

Description

파일인 경우 actor쪽에서 읽어서 시작할 수 있다.
시스템이 결과를 파일에 적는다.

Communication diagram

usecase당 바운더리 1개, 컨트롤 1개

바운더리: 파일 읽고 파일 쓰고 읽은건 컨트롤한테, 받은건 파일에 쓰면 됨

컨트롤: 필요한 엔티티 클래스 부르고 결과를 바운더리한테 전달

전달하는건 컨트롤에서 바운더리 전달인데 이걸 함수가 있는게 아니라 컨트롤 함수의 리턴값을 바운더리가 바로 받는 것

따라서 번호가 2번으로 안넘어감. 1 -> 1.1 -> 1.1.1 이런식임

컨트롤 바운더리는 Usecase 3개니까 몇개?

Entity는 몇개?

설문이 여러 개가 될 수 있음 따라서 각 설문의 응답도 여러개가 될 수 있다.

따라서 "이 설문에 응답이 여러개 있을 수 있다" 라는 관계가 표현이 되어 함 -> 클래스 다이어그램에서 일대다면 컬렉션 클래스까지 만들어보자

Design class diagram

public/private , return타입, IS-A, HAS-A관계를 표현

association 관계를 그릴 필요 없음

19p에서 화살표 없애도 됨

Design class Diagram 예시

Visibility

**BankAccount class
with visibility
specified**

BankAccount
- <u>nextAccountNumber: Integer</u> - accountNumber: Integer - accountName: String {not null} - balance: Money = 0 - /availableBalance: Money - overdraftLimit: Money
+ open(accountName: String): Boolean + close(): Boolean + credit(amount: Money): Boolean + debit(amount: Money): Boolean + viewBalance(): Money # getBalance(): Money - setBalance(newBalance: Money) # getAccountName(): String # setAccountName(newName: String)

일대다면 컬렉션 클래스

One-to-many associations using a collection class

